

2026-04-18 Rainha Diagnostico Definitivo v2

DIAGNÓSTICO DEFINITIVO v2 — MÉTODO RAINHA DA APROVAÇÃO

Plano de Estabilização + Caminho para Escala (200 → 1.600 alunos)

Autor: Claude Opus 4.7, sob orientação de Mestre (ForYou Code)

Data: 2026-04-18

Versão: 2.0 — consolidada após análise crítica da v1

(diagnostico_definitivo_rainha.docx)

Status: Plano executável, não solução mágica

Destinatários: Mestre, José (dev), Dayane (cliente, partes cliente-facing)

0. AVISO HONESTO — leia antes de tudo

Este documento **não** é uma solução mágica para 1.600 alunos. É um **plano de estabilização em 4 semanas + caminho controlado para escala em 6 meses**. Tratá-lo como "arquitetura final para 1.600 usuários" seria repetir o erro da v1.

O que este plano entrega:

- Mês 1: zero bugs recorrentes voltando, backup diário, conformidade legal mínima, observabilidade real.
- Mês 2: CI/CD robusto com 8 gates, performance validada sob carga real, feature flags, runbook testado.
- Mês 3–6: migração controlada do Lovable Cloud para Supabase direto, preparação sólida para 800–1.600 alunos.

O que este plano não entrega:

- Arquitetura multi-tenant para 10.000 alunos (não é prioridade agora).
- Eliminação total de todo bug (impossível; reduz recorrência em 80–90%).
- Independência total do José (Fase 6 reduz risco, não elimina).
- Certificação ISO/SOC2 (escopo futuro, após 1.000 alunos pagantes).

Se algum ponto deste plano falhar, há **rollback documentado** em cada fase. Nenhuma mudança é destrutiva sem reversão em ≤1h (exceto cutover Lovable→Supabase, que tem

janela de 30 dias de paralelo).

1. SUMÁRIO EXECUTIVO

1.1 Situação atual — realidade sem maquiagem

Métrica	Valor real	Status
Alunos pagantes	202 (potencial 400–1.600)	Saudável mas frágil
Questões no banco	97.955	OK
Perfis totais	279	OK
Tabelas	47	OK
Edge Functions	30	Fora do pipeline CI
Bugs recorrentes/mês	134 ChunkLoadError + ~30 outros	Crítico
Alunos afetados por ChunkLoadError	48 / 202 (24%)	Crítico
Uptime estimado	~95% (sem monitoring formal)	Abaixo do aceitável
Backup do banco	INEXISTENTE	Risco #1 — catastrófico
PITR Supabase	Não habilitado	Risco #1
DPIA / conformidade LGPD	INEXISTENTE	Risco jurídico direto
Conformidade ECA Digital 2026	Não avaliada	Risco jurídico direto
Single point of failure humano	José (solo, 2.808 commits bot)	Crítico
Dependência externa crítica	Lovable Cloud (lock-in)	Médio-alto
Pico de carga (segunda 17h)	636 tentativas simultâneas	Não testado formalmente
Lovable reescrevendo código blindado	Sim, recorrente	Alto
Documentos de blindagem no repo	Sim (BLINDAGEM_FIXES.md, PLANO_CORRECAO_BUGS.md)	Existem mas não são lidos pelo Lovable

1.2 Prioridades por ordem de urgência

1. **Backup e DR do banco** (48h) — sem backup, qualquer erro = fim do projeto.
2. **Blindagem anti-regressão** (semana 1) — CLAUDE.md + AST rules impedem Lovable desfazer fixes.
3. **Frontend estável** (semana 1) — ChunkLoadError zerado com Cloudflare Pages + manualChunks + ErrorBoundary.
4. **Observabilidade real** (semana 2) — Sentry + Cloudflare Analytics + alertas Discord.
5. **CI/CD robusto** (semana 3) — 8 gates, Edge Functions no pipeline, regression suite.
6. **Conformidade LGPD/ECA** (semana 4) — DPIA + consentimento responsável + right-to-erasure.
7. **Performance e capacity** (mês 2) — load test autenticado, connection pooling, RLS otimizado.
8. **Feature flags + governance** (mês 2) — PostHog self-hosted, runbook, redução SPoF José.
9. **Migração Lovable → Supabase direto** (mês 3–5) — gatilhos explícitos, paralelo 30 dias.

1.3 Decisão arquitetural central

Manter Lovable Cloud até mês 3. Migrar para Supabase direto entre mês 3 e 5. Corte definitivo mês 6.

Justificativa detalhada na seção 4.1.

2. DIAGNÓSTICO TÉCNICO REAL

2.1 Stack verificada (não "stack imaginada")

Frontend:

- React 18.3.1 + TypeScript 5.8.3 + Vite 5.4.19 (SWC)
- Tailwind 3.4 + shadcn/ui + Radix UI completo
- TanStack React Query 5.83 + React Hook Form 7.61 + Zod 3.25
- React Router 6.30.1
- jsPDF + autotable + html2canvas (PDFs, provas, redações)
- KaTeX (fórmulas matemáticas)
- @dnd-kit (reordenação do planner)
- vite-plugin-pwa (autoUpdate)

Backend (via Lovable Cloud hoje):

- Supabase — 47 tabelas, 30 Edge Functions
- RLS com helpers: `has_role()`, `is_teacher_of_student()`, `auth.uid()`
- Perfis: enum `app_role` (student, teacher, director) + admin separado
- Pagamentos: Eduzz via Edge Function `eduzz-webhook`
- Email: Resend via Edge Function
- IA: Lovable AI Gateway (Gemini 2.5 Flash/Pro + GPT-5, 30 créditos/mês)

Deploy atual:

- Vercel (frontend)
- GitHub: `ynwwilson/metodorainha-dc07fc66`
- 2.808 commits, 100% via `lovable-dev[bot]`

Volumes reais (censo 2026-04):

- 202 assinaturas ativas
- 97.955 questões catalogadas
- 279 perfis
- Pico segunda 17h: 636 tentativas simultâneas em `student_question_attempts`

2.2 Root cause dos bugs recorrentes (análise profunda)

Os 30 bugs não voltam por incompetência do José — voltam por **ausência de guardrails estruturais**:

Bug #1: ChunkLoadError (134/mês, 24% dos alunos)

- Causa 1: Vite sem `manualChunks` → bundle fragmentado imprevisível por hash.
- Causa 2: Vercel com cache agressivo e `_headers` implícitos → browser pega `index.html` novo apontando para chunks antigos.
- Causa 3: ErrorBoundary não detecta chunk errors, não faz retry.
- Fix parcial (só um dos três) = não resolve. Precisa dos três simultâneos + Cloudflare Pages.

Bug #2–6: Cache inválido / invalidateQueries ausente (React Query)

- Padrão: `useMutation` sem `onSuccess: () => queryClient.invalidateQueries(...)`.
- Sintoma: usuário vê dado stale após ação.
- Fix recorrente porque Lovable reescreve sem saber do padrão.

- Solução real: ESLint AST rule custom bloqueando `useMutation` sem `invalidateQueries` no CI.

Bug #7–12: RLS parcial e gaps de segurança

- `questions` com `SELECT true` para qualquer autenticado → scraping livre de 97.955 questões.
- `simulations` com `SELECT true` → aluno vê simulado de turma alheia.
- `ai_explanation_cache` totalmente aberta → vazamento de explicações de IA (custo + privacidade).
- Fix v1 usa subquery correlata em `questions` (hot path com 97.955 rows + 636 concurrent) **sem EXPLAIN ANALYZE** — risco real de degradar performance.
- Solução correta: função SQL `is_active_subscriber(uid)` com `STABLE SECURITY DEFINER` + `EXPLAIN` obrigatório antes do deploy.

Bug #13: INC-04 — Webhook Eduzz duplicado

- Ausência de idempotência.
- Solução: tabela `eduzz_webhook_events` com unique constraint em `event_id` + upsert pattern + teste de contrato no CI.

Bug #14: INC-10 — Realtime subscription leak

- Componentes usando `supabase.channel()` sem `cleanup` no `useEffect`.
- Solução: ESLint rule `require-channel-cleanup` + auditoria manual inicial.

Bug #15: Trigger `on_answer_inserted` parcial

- Em alguns casos não atualiza XP/streak.
- Solução: auditar trigger, adicionar teste de integração no CI.

Bug #16–30: Soma correções de somatória, busca semântica, revisões SRS, caderno de erros, sala de estudos, auth em Edge Functions.

- Todos documentados em `BLINDAGEM_FIXES.md` no repo.
- Todos voltam porque Lovable não lê o arquivo.
- **Solução sistêmica:** `CLAUDE.md` na raiz do repo (Lovable lê automaticamente) listando cada BUG-XX com fix aplicado + ESLint rules validando no CI.

2.3 O que estava BOM no plano v1 (mantido com ajustes)

- Diagnóstico central "Lovable é o inimigo ativo do código estável" — correto e mantido.

- **Decisão de não migrar banco imediatamente** — correto, mantido.
- **CLAUDE.md como régua de governança** — excelente, mantido e expandido.
- **ErrorBoundary com retry em chunk errors** — correto, mantido.
- **Migração para Cloudflare Pages** — correto, mantido.
- **Separação dos 7 blocos de missão do prompt Grok** — útil, mantido como apêndice.
- **Honestidade parcial em honestidade_qa_rainha.docx** — raro e valorizado.
- **Estrutura de fases** — mantida, mas re-temporizada.

2.4 O que estava FRACO no plano v1 (corrigido na v2)

Ponto fraco v1	Correção v2
Backup ausente	Fase 0 prioridade absoluta (48h)
Gates por <code>grep</code>	ESLint AST rules custom + <code>@typescript-eslint/parser</code>
Smoke test em produção contra conta real	Staging separado + conta QA com <code>is_qa_account</code> isolada por RLS
RLS fix com subquery correlata sem EXPLAIN	Função SQL <code>is_active_subscriber()</code> STABLE + EXPLAIN obrigatório
k6 só em homepage <code>http.get</code>	k6 autenticado simulando fluxo crítico (aluno respondendo questão)
Session replay sem DPIA	Session replay bloqueado até Fase 4 concluída + masking agressivo
Edge Functions fora do pipeline	Todas com contract test no CI (Gate 7)
"Manter Lovable Cloud permanentemente"	Plano de saída mês 3–6 com gatilhos explícitos
Timing 23h	4 semanas estabilização + 2–5 meses escala (realista)
SPoF José	Fase 6: runbook, pair programming, documentação completa
Ausência de feature flags	Fase 6: PostHog self-hosted + 5 flags iniciais
Ausência de DPIA/ECA Digital	Fase 4 inteira dedicada
Imagens no Supabase Storage (caro em escala)	Fase 5: migração para Cloudflare R2 + CDN
Connection pooling não endereçado	Fase 5: Supervisor transaction mode
Ausência de runbook	Fase 6: runbook testado em 3 simulações
Ausência de disaster recovery test	Fase 0: verify semanal + DR test mensal a partir do mês 2

3. OBJETIVO FINAL E CRITÉRIOS DE SUCESSO

3.1 Visão aos 6 meses (outubro 2026)

Rainha da Aprovação rodando com:

- 400–800 alunos pagantes (escalando conforme venda da Dayane)
- Supabase direto (Lovable Cloud cortado)
- Backup diário automatizado + PITR + DR test mensal
- DPIA concluído, termos atualizados, consentimento responsável ativo
- Zero incidentes LGPD
- Uptime > 99.5%
- Bugs recorrentes < 5/mês
- Pico segunda 17h absorvido com p95 < 500ms no endpoint crítico
- Pipeline CI/CD com 8 gates, incluindo Edge Functions
- Feature flags ativos, deploys desacoplados de releases
- Runbook testado, Mestre consegue dar primeiro diagnóstico em incidentes P0 sem depender do José
- Documentação técnica completa no vault (Stark)

3.2 KPIs mensuráveis

KPI	Baseline atual	Meta mês 1	Meta mês 3	Meta mês 6
ChunkLoadError/mês	134	< 10	< 3	0
Bugs recorrentes/mês	~30	< 10	< 5	< 3
p95 resposta caderno	Desconhecido	< 800ms	< 500ms	< 400ms
p95 resposta <code>answer_question</code> (POST)	Desconhecido	< 600ms	< 400ms	< 300ms
Uptime	~95%	> 98%	> 99%	> 99.5%
Backup age máximo	∞	< 24h	< 6h	< 1h (PITR)
Tempo médio detecção incidente	Usuário avisa	< 15min	< 5min	< 2min
MTTR incidente P0	Desconhecido	< 4h	< 1h	< 30min
Cobertura regression suite	0%	50%	80%	90%
Alunos afetados/incidente médio	Desconhecido	< 10%	< 5%	< 2%

KPI	Baseline atual	Meta mês 1	Meta mês 3	Meta mês 6
% deploys com rollback	Desconhecido	< 10%	< 5%	< 2%

3.3 Definition of Done por fase

Cada fase só é considerada concluída quando **todos** os critérios são atendidos:

1. Código em produção estável por > 7 dias sem regressão detectada.
2. Documentação no vault atualizada (nota datada em `stark/Stark/AI-Brain/`).
3. Mestre executou validação manual conforme checklist da fase.
4. Métricas da fase dentro do alvo.
5. Rollback documentado e testado em staging.

4. DECISÕES ARQUITETURAIS COM JUSTIFICATIVA

4.1 Lovable Cloud — manter agora, sair entre mês 3–6

Decisão: manter como backend até o mês 3. Migrar completamente entre mês 3 e 5. Corte definitivo no mês 6.

Por que manter agora (não migrar imediatamente):

- Migrar no mês 1 com 202 assinaturas ativas, 97.955 questões e José sem bandwidth = suicídio.
- O problema imediato (ChunkLoadError + regressões) é resolvível sem tocar no banco.
- Supabase via Lovable é Supabase — schema e Edge Functions são os mesmos; migração é re-apontar credenciais e reconstruir deploy pipeline.

Por que migrar no mês 3–5 (não "eventualmente"):

- Lovable Cloud é camada intermediária cobrada, com acesso CLI limitado, sem SLA formal.
- Lovable já desconectou GitHub e criou repo novo no passado (screenshots confirmam) — risco real.
- Para 1.600 alunos, cada camada extra de latência e cada ponto de decisão externo é custo.
- Permite Supabase CLI completo, MCP sem gambiarra, migrations versionadas, branching, Edge Functions deploy por git.

Gatilhos de saída antecipada (qualquer um dispara migração imediata, pulando para Fase 7):

1. Lovable aumenta preço > 30%.
2. SLA Lovable degrada > 1% incidentes/mês afetando Rainha.
3. Lovable desconecta GitHub repo novamente ou corrompe schema.
4. Rainha passa de 500 alunos ativos antes do mês 3.
5. Necessidade de feature não suportada (replicação lógica, branching, etc.).

Gatilhos de adiar migração (qualquer um dispara pause):

1. Migração em staging falha 2 vezes consecutivas.
2. José ausente > 15 dias úteis durante janela de migração.
3. Incidente P0 aberto não resolvido.

4.2 Frontend — Vercel → Cloudflare Pages (Fase 1)

Por que migrar:

- Cloudflare Pages + `_headers` corretos + `manualChunks` = resolve 90% do `ChunkLoadError`.
- Bandwidth ilimitado (Vercel cobra acima do limite free).
- WAF grátis (Vercel não oferece no plano base).
- Workers para edge logic (rate limit, geofencing, A/B test).
- Bot Fight Mode + Analytics grátis.
- Preview deployments por branch com URLs únicas.

Custo de migração: 2–4h.

4.3 Cloudflare Workers (Fase 2 em diante)

Uso recomendado:

- Rate limit por IP + `user_id` (protege scraping do banco de questões — risco real com concorrentes do nicho).
- Geofencing Brasil-only (reduz superfície de ataque bot).
- A/B test edge sem JS extra no cliente.
- Cache de respostas de Edge Functions stateless (`/health` , `/questions-count`).

NÃO usar Workers para:

- Lógica de negócio — fica em Edge Functions Supabase.
- State management — não suportam.

- Qualquer coisa que dependa de cold-start determinístico.

4.4 Feature Flags — PostHog self-hosted (Fase 6)

Decisão: PostHog self-hosted em VPS pequeno (Hetzner CX21 ~€5.83/mês ou Contabo VPS S ~€6.99/mês).

Alternativa minimal (se VPS for adiado): Cloudflare Workers KV + pequena API interna.

Por quê:

- Deploy desacoplado de release (flag off = feature invisível).
- Rollout gradual (10% → 50% → 100%).
- Kill switch instantâneo em incidente.
- Self-hosted evita lock-in adicional e resolve LGPD (dados no BR/EU).
- PostHog também dá product analytics, session replay com masking — redundante com Sentry mas útil para funil.

Flags iniciais obrigatórios:

- `new-caderno-ui` — rollout gradual UI nova.
- `ai-explanation-v2` — nova versão explicação IA.
- `advanced-planner` — planner com `@dnd-kit` melhorado.
- `kill-switch-eduzz` — desliga webhook Eduzz emergência.
- `kill-switch-signup` — bloqueia cadastros novos emergência.
- `kill-switch-ai` — desliga chamadas AI Gateway emergência.

4.5 MCPs e Automação

MCPs a configurar no Claude Code (tanto José quanto Mestre):

MCP	Uso	Fase
Supabase MCP	Query, migrations, RLS management, EXPLAIN ANALYZE	Fase 0
GitHub MCP	PRs, issues, Actions logs, branch management	Fase 1
Playwright MCP	Execução de regression suite local antes de commit	Fase 3
Sentry MCP	Ler incidentes, stack traces dentro do Claude	Fase 2
Cloudflare MCP	Deploy, logs, WAF, Workers	Fase 1
Filesystem MCP	Vault Obsidian (já ativo)	—
Obsidian MCP	Ler/escrever notas no vault (já ativo, com fallback)	—

Supabase Agent Skills (habilitar assim que sair do beta pago):

- Skill `rainha-db-audit` — escaneia RLS, índices, dead tuples, bloat, prepara relatório semanal.
- Skill `rainha-edge-deploy` — deploy de Edge Function com teste de contrato antes.
- Skill `rainha-backup-verify` — checa integridade do último dump restaurando em banco ephemeral.
- Skill `rainha-rls-explain` — gera EXPLAIN ANALYZE de todas as policies em hot path.

4.6 Guardrails modernos de vibe coding

Guardrail	Ferramenta	O que bloqueia
ESLint AST rules custom	<code>eslint-plugin-rainha-blindagem</code> (próprio)	Padrões proibidos: <code>mocks</code> em prod, <code>useMutation</code> sem <code>invalidateQueries</code> , canal realtime sem cleanup, <code>Zod</code> ausente em boundary
TypeScript strict + <code>noUncheckedIndexedAccess</code>	<code>tsconfig</code>	Bugs de <code>undefined</code> em arrays e objetos
<code>Zod</code> em toda boundary	Runtime validation	Dados inválidos atravessando camadas
Pre-commit hook local	Husky + lint-staged	Commit com lint/type error
CI gates (8)	GitHub Actions	Merge de código quebrado, Edge Function sem contract test
Staging environment	Supabase project separado	Teste em produção
Feature flags	PostHog self-hosted	Deploy arriscado sem escape
Runbook + alerts	Sentry + Cloudflare + Discord	Incidente invisível
CLAUDE.md obrigatório	Root do repo Rainha	IA reescrevendo código blindado
Conta QA isolada	RLS <code>is_qa_account</code> flag	Contaminação de dados de produção
DPIA + consent ledger	Tabela <code>consent_events</code>	Incidente LGPD sem prova

5. PLANO DE EXECUÇÃO — 7 FASES

Visão geral do cronograma

ABR 2026	Sem 1 [Fase 0] Backup + DR	⚠️ 48h críticas
	Sem 1 [Fase 1] Frontend + CLAUDE.md	(paralelo a Fase 0)
	Sem 2 [Fase 2] Observabilidade	
	Sem 3 [Fase 3] CI/CD robusto	
	Sem 4 [Fase 4] Conformidade LGPD/ECA	
MAI 2026	Sem 1-2 [Fase 5] Performance & escala	
	Sem 3-4 [Fase 6] Feature flags + governance	
JUN-AGO	[Fase 7] Migração Lovable → Supabase direto staging → paridade → replicação → cutover → paralelo 30d → corte	
SET 2026	Rainha 100% Supabase direto, 800 alunos suportados, todas métricas no verde	

FASE 0 — SEGURANÇA IMEDIATA ⚠️ (48h)

Objetivo: não perder dados. Sem isso, qualquer outra fase é irrelevante.

5.0.1 Tarefas manuais do Mestre (~2h)

1. Criar bucket Cloudflare R2:

- Login Cloudflare → R2 → Create bucket `rainha-backups`
- Region: automatic
- Versioning: enabled
- Object lifecycle: retain 90 dias, archive após 30

2. Gerar tokens Cloudflare R2:

- My Profile → API Tokens → Create Token
- Permissions: R2:Edit
- Copiar: `R2_ACCESS_KEY_ID` , `R2_SECRET_ACCESS_KEY` , `R2_ACCOUNT_ID`

3. Habilitar PITR no Supabase:

- Dashboard → Settings → Database → Point in Time Recovery
- Retenção mínima: 7 dias
- Upgrade de plano se necessário (+ US\$25-100/mês, depende do tier)

4. Coletar credenciais Supabase:

- Service Role Key (para backup script)
- Database URL direct connection (formato `postgresql://...:6543`)

- Project Ref

5. Configurar GitHub Secrets:

- Repo → Settings → Secrets and variables → Actions
- Adicionar: SUPABASE_DB_URL , SUPABASE_SERVICE_ROLE_KEY , R2_ACCESS_KEY_ID , R2_SECRET_ACCESS_KEY , R2_ACCOUNT_ID , R2_BUCKET

6. Enviar credenciais ao Claude Code (colar em seção protegida do vault ou mensagem cifrada).

5.0.2 Tarefas automáticas do Claude Code (~4h)

1. Backup inicial imediato:

- Rodar `pg_dump --no-owner --no-privileges --format=custom` contra produção agora.
- Armazenar em dois lugares: local (`C:\Users\ynwwi\Backups\rainha-<data>.dump`) + R2.
- Validar integridade: `pg_restore --list` retorna estrutura esperada.

2. GitHub Action `.github/workflows/backup-daily.yml` :

- Cron: `0 6 * * *` (diário 03h BRT).
- Steps: `pg_dump` → `gzip` → upload R2 com retention tag → notificar Discord em falha.
- Rotation: R2 lifecycle rule (não via action).

3. Script `scripts/backup-manual.sh` :

- Dump imediato sob demanda (antes de migration grande).
- Uso: `./scripts/backup-manual.sh "pre-migration-fix-rls"`.

4. GitHub Action `.github/workflows/backup-verify.yml` :

- Cron: `0 5 * * 0` (semanal, domingo 02h BRT).
- Baixa último backup → restaura em banco ephemeral Postgres Docker → roda queries sanity check (`SELECT count(*) FROM profiles , FROM questions , etc.`) → reporta no vault + Discord.

5. Auditoria webhook Eduzz (INC-04):

- Query `edge_logs + subscriptions` buscando duplicatas nos últimos 90 dias.
- Gerar relatório: quantos eventos duplicados, valor financeiro implicado, usuários afetados.
- Salvar em `stark/Stark/AI-Brain/2026-04-18-auditoria-eduzz.md`.

6. Runbook inicial `stark/Stark/knowledge/rainha-runbook-fase0.md` :

- "Como restaurar backup em incidente P0" (passo-a-passo testado).
- "Como fazer `pg_dump` manual agora mesmo".
- "Como contactar suporte Supabase".

5.0.3 Deliverables da Fase 0

- ☐ Backup inicial completo em R2 + local (Mestre verifica hash).
- ☐ Backup diário automatizado (primeira execução validada).
- ☐ Backup verify semanal (primeiro relatório OK).
- ☐ PITR habilitado Supabase.
- ☐ Relatório auditoria Eduzz salvo no vault.
- ☐ Runbook inicial no vault.

5.0.4 Rollback

Nenhuma mudança destrutiva. Único risco: custo R2 + PITR ($\leq$ US\$50/mês). Reversível cancelando billing.

FASE 1 — BLINDAGEM FRONTEND + CLAUDE.md (semana 1, paralela à Fase 0)

Objetivo: matar ChunkLoadError e impedir Lovable de reescrever código blindado.

5.1.1 Tarefas manuais do Mestre (~1h)

1. Cloudflare Pages:

- Login Cloudflare → Pages → Create project.
- Connect to Git → repo `ynwwilson/metodorainha-dc07fc66`.
- Build command: `npm run build`.
- Build output: `dist`.
- Env vars: copiar do Vercel (`VITE_SUPABASE_URL` , `VITE_SUPABASE_ANON_KEY` , etc.).

2. DNS:

- Após primeiro deploy Cloudflare OK e estável >48h, apontar domínio para Cloudflare Pages.
- Manter Vercel paralelo por 7 dias como fallback.

3. Cloudflare Security baseline:

- WAF: OWASP Core Rule Set ON.
- Bot Fight Mode: ON.
- Rate Limiting: regra 100 req/min por IP no path `/auth/*` + 300 req/min no `/rest/*`.
- SSL/TLS: Full (strict).
- Always Use HTTPS: ON.
- Automatic HTTPS Rewrites: ON.

5.1.2 Tarefas automáticas do Claude Code (~12h)

1. vite.config.ts com manualChunks:

```
build: {
  rollupOptions: {
    output: {
      manualChunks: {
        'react-vendor': ['react', 'react-dom', 'react-router-dom'],
        'supabase': ['@supabase/supabase-js'],
        'ui-radix': [/* lista completa Radix */],
        'data-fetching': ['@tanstack/react-query'],
        'forms': ['react-hook-form', 'zod', '@hookform/resolvers'],
        'pdf': ['jspdf', 'jspdf-autotable', 'html2canvas'],
        'katex': ['katex'],
        'dnd': ['@dnd-kit/core', '@dnd-kit/sortable'],
      },
      chunkFileNames: 'assets/[name]-[hash].js',
      entryFileNames: 'assets/[name]-[hash].js',
      assetFileNames: 'assets/[name]-[hash][extname]',
    },
  },
  chunkSizeWarningLimit: 800,
  sourcemap: true, // upload Sentry
}
```

2. src/components/AppErrorBoundary.tsx :

- Classe React com state { hasError, isChunkError, retryCount, error }.
- getDerivedStateFromError detecta: "dynamically imported module", "Failed to fetch", "MIME type", "Importing a module script failed".
- componentDidCatch: se chunk error e retryCount === 0, setTimeout(() => window.location.reload(), 1500) e incrementa.
- Log para Sentry com tag chunk_error: true.
- Fallback UI amigável se retry falhar (botão "tentar novamente" + link suporte).

3. public/_headers Cloudflare Pages:

```
/assets/*
  Cache-Control: public, max-age=31536000, immutable

/index.html
  Cache-Control: public, max-age=0, must-revalidate

/*
  X-Frame-Options: DENY
  X-Content-Type-Options: nosniff
  Referrer-Policy: strict-origin-when-cross-origin
```

```
Permissions-Policy: camera=(), microphone=(), geolocation=()
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

4. **CLAUDE.md na raiz do repo Rainha** (anexo A deste documento). Estrutura:

- Identidade (Rainha é SaaS de Dayane, menores envolvidos, LGPD crítico).
- Stack obrigatória.
- 12 REGRAS ABSOLUTAS (não reescrever código blindado, `invalidateQueries` obrigatório, Zod em boundary, etc.).
- Workflow obrigatório (ler `BLINDAGEM_FIXES.md` + `PLANO_CORRECAO_BUGS.md` antes de tocar em código correlato).
- Lista de BUG-01..BUG-30 + INC-01..INC-12 com fix aplicado.
- Ao final de cada sessão, atualizar `CHANGELOG-SESSAO.md`.

5. **ESLint plugin** `eslint-plugin-rainha-blindagem`:

- Rule `no-mutation-without-invalidate` — detecta `useMutation` sem `onSuccess` contendo `invalidateQueries`. AST-based usando `@typescript-eslint/parser`.
- Rule `no-mock-in-production` — bloqueia `MockData`, `FAKE_`, `TODO_DEV_ONLY`, `__DEV MOCK__`.
- Rule `no-console-in-prod-paths` — `console.log` fora de `src/lib/debug/`.
- Rule `require-zod-at-boundary` — Edge Functions + form handlers precisam de `.parse()` ou `.safeParse()`.
- Rule `require-channel-cleanup` — `supabase.channel()` dentro de `useEffect` exige `cleanup removeChannel`.
- Rule `no-direct-supabase-in-component` — queries Supabase só em `src/lib/api/` ou hooks dedicados.

6. **Husky + lint-staged pre-commit:**

- `npm run lint` (ESLint + custom rules).
- `npx tsc --noEmit`.
- Se falhar, bloqueia commit localmente.

7. **Commit tudo via José** (não via Lovable) para garantir que CLAUDE.md está no repo antes da próxima sessão Lovable.

5.1.3 Deliverables da Fase 1

- ☐ Deploy estável Cloudflare Pages > 48h.
- ☐ `ChunkLoadError` reduzido > 80% (Sentry comprova).
- ☐ `CLAUDE.md` na raiz do repo, commitado.
- ☐ ESLint custom rules funcionando (teste: PR com `useMutation` sem `invalidate` → bloqueia).
- ☐ Pre-commit hook ativo no ambiente José.
- ☐ Vercel online como fallback por 7 dias.

5.1.4 Rollback

- DNS revert para Vercel (< 10min).
 - ErrorBoundary é aditivo — remover não quebra nada.
 - `manualChunks` remover = volta ao estado atual (bundle único).
-

FASE 2 — OBSERVABILIDADE (semana 2)

Objetivo: saber o que está acontecendo em produção em tempo real. Sem métricas, otimização é chute.

5.2.1 Tarefas manuais do Mestre (~1h)

1. Conta Sentry:

- Criar org `foryou-code` / projeto `rainha`.
- Plano Free (100k events/mês — cobre 200-400 alunos).
- Gerar DSN + Auth Token.

2. GitHub Secrets:

- `SENTRY_DSN`, `SENTRY_AUTH_TOKEN`, `SENTRY_ORG`, `SENTRY_PROJECT`.

3. Discord webhook para alertas:

- Criar canal `#rainha-alerts`.
- Webhook URL → GitHub Secret `DISCORD_ALERTS_WEBHOOK`.
- Alternativa: Telegram bot via Composio.

4. Conta Checkly (opcional, free tier cobre 10k checks/mês):

- Para health checks de 5min (não smoke em prod).

5.2.2 Tarefas automáticas do Claude Code (~8h)

1. `src/lib/sentry.ts` LGPD-safe (antes do DPIA):

```
Sentry.init({
  dsn: SENTRY_DSN,
  environment: import.meta.env.MODE,
  release: import.meta.env.VITE_COMMIT_SHA,
  tracesSampleRate: PROD ? 0.1 : 1.0,
  // SESSION REPLAY DESABILITADO até DPIA Fase 4
  replaysSessionSampleRate: 0,
  replaysOnErrorSampleRate: 0,
  integrations: [
    Sentry.browserTracingIntegration(),
    supabaseIntegration(supabase, { tracing: true, breadcrumbs: true,
```

```

errors: true }},
],
beforeSend(event, hint) {
  // Remove PII
  if (event.user) {
    delete event.user.email;
    delete event.user.ip_address;
    event.user.id = hashUserId(event.user.id); // hash determinístico
  }
  // Remove PII em extra / contexts
  sanitize(event.extra);
  sanitize(event.contexts);
  return event;
},
ignoreErrors: [
  'ResizeObserver loop limit exceeded',
  'ResizeObserver loop completed with undelivered notifications',
  /extension:\\/,
  /chrome-extension:\\/,
  /moz-extension:\\/,
],
denyUrls: [
  /chrome-extension:\\/,
  /extensions\\/,
],
});

```

2. Source maps upload CI:

- Vite plugin @sentry/vite-plugin.
- Upload em cada deploy com release = commit SHA.

3. Alertas Sentry:

- chunk_error > 5 em 10min → Discord.
- Qualquer erro em eduzz-webhook Edge Function → Discord + email Mestre.
- Erro não categorizado > 50/hora → Discord.
- P0: erro impedindo login > 10 em 5min → Discord + email + Telegram.
- Regression: erro com tag bug_id: BUG-XX que já foi "fixed" → Discord P0.

4. Cloudflare Analytics Dashboard:

- Habilitar Web Analytics (grátis, sem cookies → LGPD-friendly).
- Dashboard custom: p95 TTFB, cache hit rate, top paths 4xx/5xx, bot traffic.

5. Supabase logs estruturados:

- Edge Functions com logger padronizado:

```

function log(level: 'info'|'warn'|'error', msg: string, ctx:
Record<string, any>) {

```

```

    console.log(JSON.stringify({
      level,
      timestamp: new Date().toISOString(),
      request_id: ctx.request_id ?? crypto.randomUUID(),
      user_id: ctx.user_id ? hashUserId(ctx.user_id) : null,
      msg,
      ...ctx,
    }));
  }
}

```

- Query helper com log de queries > 500ms (pg_stat_statements já ativo).

6. Health endpoint real supabase/functions/health/index.ts :

- Checa: DB connection, Eduzz webhook responsive, AI Gateway responsive, Storage responsive.
- Retorna 200 com { status, checks: {...}, latency }.
- Checkly ping a cada 5min.

7. Painel unificado no vault:

- stark/Stark/AI-Brain/rainha-dashboard-links.md com links para Sentry, Cloudflare Analytics, Supabase logs, Checkly, PostHog (após Fase 6).

5.2.3 Deliverables da Fase 2

- ☐ Sentry recebendo eventos de produção com sanitização LGPD.
- ☐ Source maps funcionando (stack traces legíveis).
- ☐ Alertas Discord disparando (teste: forçar erro → alerta chega em < 30s).
- ☐ Cloudflare Analytics com 48h+ de baseline.
- ☐ Health endpoint respondendo, Checkly monitorando.
- ☐ Runbook de alertas (quando dispara X, fazer Y).

5.2.4 Riscos e mitigações

- **Sentry free estoura** se houver bug novo massivo. Mitigar: sampling + ignoreErrors agressivo + alert em Sentry usage > 80%.
- **LGPD ainda pendente** nesta fase. Por isso session replay desligado. Fase 4 libera.

FASE 3 — CI/CD ROBUSTO (semana 3)

Objetivo: impossibilitar merge de código quebrado ou que regride bugs conhecidos.

5.3.1 Tarefas manuais do Mestre (~30min)

1. GitHub branch protection:

- Repo → Settings → Branches → `main`.
- Require PR before merging: ✓
- Require status checks: ✓ (marcar os 8 gates após primeira execução).
- Require branches up to date: ✓
- Do not allow bypass: ✓ (nem admin).
- Restrict pushes to main: ✓

2. Dependabot / Renovate:

- Habilitar Dependabot security updates (min).

5.3.2 Tarefas automáticas do Claude Code (~16h)

1. GitHub Actions `.github/workflows/ci.yml` — 8 gates:

Gate 1 — Lint:

- `npm run lint` (ESLint + `eslint-plugin-rainha-blindagem`).
- Falha em qualquer error; warnings permitidos mas listados.

Gate 2 — TypeCheck:

- `npx tsc --noEmit` com `tsconfig strict + noUncheckedIndexedAccess`.

Gate 3 — AST Blindagem (script custom):

- `scripts/check-blindagem.mjs` usando `@typescript-eslint/parser`.
- Detecta padrões proibidos via AST (não `grep`).
- Exemplos: chamadas a `supabase.from()` fora de `src/lib/api/`; imports de arquivos marcados com `@blindado`; mutação direta de `queryClient` sem hook.

Gate 4 — Build:

- `npm run build`.
- Falha se bundle > 2MB total ou chunk > 800KB.
- Gera artefatos para Gate 6.

Gate 5 — Unit tests (Vitest):

- `npm run test:unit`.
- Cobertura mínima 60% em `src/lib/`.

Gate 6 — Regression suite (Playwright):

- `npm run test:regression`.
- 30 testes, um por bug histórico.

Gate 7 — Edge Functions contract tests:

- Cada Edge Function em `supabase/functions/*/` tem `test.ts`.
- Deno test roda em CI contra function local + mock Supabase.
- `eduzz-webhook.test.ts` **obrigatório** testa: idempotência, assinatura válida, payload malformado, replay attack.

Gate 8 — Migration check:

- Se houver mudança em `supabase/migrations/`, roda migration em banco ephemeral Postgres + verifica que RLS continua ativo em tabelas críticas.
- Verifica presença de `CREATE INDEX CONCURRENTLY` em migrations grandes.

2. `.github/workflows/deploy.yml`:

- Trigger: push em `main`, após todos os gates.
- Deploy Cloudflare Pages via action oficial.
- Deploy Edge Functions via Supabase CLI: `supabase functions deploy --project-ref $REF`.
- Upload source maps Sentry.
- Post-deploy smoke test em **staging** (não em prod).
- Notificação Discord: `Deploy #XXX OK, commit abc123, changelog: ...`.

3. Staging environment:

- Supabase project `rainha-staging` separado.
- Schema clone de produção, dados sintéticos (100 questões, 5 perfis QA, zero PII real).
- Cloudflare Pages preview deployments apontam para staging.
- URL: `rainha-staging.foryoucode.app`.

4. Conta QA com isolamento RLS:

```
ALTER TABLE profiles ADD COLUMN is_qa_account BOOLEAN DEFAULT FALSE;

-- Política para não contar QA em métricas de negócio
CREATE POLICY "exclude_qa_from_stats" ON profiles
  FOR SELECT USING (
    is_qa_account = FALSE OR current_setting('app.include_qa', true) =
    'true'
  );
```

5. Playwright regression suite `tests/regression/`:

- `bug-01-somatoria-vazia.spec.ts`
- `bug-02-chunk-error.spec.ts`

- `bug-03-subjects-undefined.spec.ts`
- ...
- `bug-30-webhook-eduzz-duplicado.spec.ts`
- Cada teste: bug original (link CLAUDE.md#BUG-XX), repro steps, assertion.
- Retries: 2 (flaky mitigation).
- Fixtures determinísticas via `supabase/functions/test-seed/`.

6. Edge Functions contract tests:

- Fixtures em `supabase/functions/*/test/fixtures/`.
- `eduzz-webhook` testa: payload real assinado, payload inválido, payload duplicado (idempotência), replay attack com timestamp antigo.
- Gate 7 só passa se 100% dos testes passarem.

5.3.3 Deliverables da Fase 3

- ☐ 8 gates rodando em todo PR (primeiro PR de teste valida).
- ☐ Staging funcional.
- ☐ Regression suite com 30 testes verdes.
- ☐ Edge Functions com contract tests, 100% passando.
- ☐ Branch protection ativa; impossível push direto em main.

5.3.4 Riscos e mitigações

- **Playwright flaky em CI:** retries + ambiente headless estável + fixtures determinísticas + waits explícitos.
- **Staging diverge de prod:** script semanal `scripts/sync-staging-schema.mjs` roda em Action.
- **CI lento:** paralelizar gates independentes, cache de `node_modules` e `~/.cache/ms-playwright`.

FASE 4 — CONFORMIDADE LGPD + ECA DIGITAL 2026 (semana 4)

Objetivo: Rainha legalmente defensável. Risco jurídico zerado para Mestre e Dayane.

Contexto crítico: Rainha atende menores de idade. LGPD (art. 14) exige consentimento específico do responsável para menores de 12. ECA Digital 2026 endurece ainda mais para adolescentes. Multa LGPD até R\$50M ou 2% do faturamento. Ignorar é risco existencial.

5.4.1 Tarefas manuais do Mestre + advogado (~10h)

1. Contratar advogado LGPD/digital (se não houver):

- Escopo: revisar Termos de Uso, Política de Privacidade, Termo de Consentimento do Responsável, DPIA.
- Custo estimado: R\$2.000–5.000 one-shot.
- Recomendação: buscar quem já atendeu EdTech.

2. DPIA (Relatório de Impacto à Proteção de Dados):

- Template oficial ANPD.
- Claude Code prepara rascunho; advogado revisa e valida.
- Seções: finalidade, dados coletados, base legal, riscos, mitigações, medidas de segurança, canais de exercício de direitos.
- Armazenar em `stark/Stark/Projetos/rainha-dpia-v1.pdf` + backup R2.

3. Nomear Encarregado de Dados (DPO):

- Pode ser Mestre inicialmente.
- Email público obrigatório: criar `dpo@metodorainhadaaprovacao.com.br`.
- Publicar em `/privacidade` com nome e contato.

4. Comunicação com Dayane:

- Apresentar mudanças: consentimento responsável obrigatório, fluxo de opt-in, direito à portabilidade/exclusão.
- Alinhar: menor < 12 anos → consentimento explícito responsável; 12–18 → modelo híbrido.
- Antecipar impacto em conversão (-5 a -15%) e compensação via UX clara.

5. Decidir estratégia de consentimento para base atual (202 alunos):

- Opção A (mais segura): re-consentir todos via email com deadline 30 dias.
- Opção B (mais pragmática): consentimento implícito via aceite na próxima sessão + banner.
- **Recomendação:** A para menores, B para maiores de 18.

5.4.2 Tarefas automáticas do Claude Code (~12h)

1. Schema update (migration):

```
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS date_of_birth DATE;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS is_minor BOOLEAN GENERATED
ALWAYS AS (date_of_birth > (now() - interval '18 years'))::date) STORED;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS guardian_email TEXT;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS guardian_name TEXT;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS guardian_document TEXT;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS consent_given_at TIMESTAMPTZ;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS consent_version TEXT;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS data_processing_consent JSONB
DEFAULT '{}'::jsonb;
```

```

CREATE TABLE IF NOT EXISTS consent_events (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  event_type TEXT NOT NULL CHECK (event_type IN (
    'consent_given', 'consent_revoked',
    'guardian_consent_requested', 'guardian_consent_given',
    'data_export_requested', 'data_export_delivered',
    'data_deletion_requested', 'data_deletion_executed'
  )),
  consent_version TEXT,
  ip_address INET,
  user_agent TEXT,
  metadata JSONB DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_consent_events_user ON consent_events(user_id, created_at
DESC);

CREATE TABLE IF NOT EXISTS terms_versions (
  version TEXT PRIMARY KEY,
  terms_url TEXT NOT NULL,
  privacy_url TEXT NOT NULL,
  effective_from TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

```

2. Fluxo de consentimento do responsável:

- Signup: se `is_minor = true`, envia email para `guardian_email` com link assinado (JWT 72h).
- Responsável clica → Edge Function `verify-guardian-consent` valida JWT → insere `consent_events` + marca `consent_given_at`.
- Sem consentimento em 72h → conta bloqueada (não deletada).
- Reminder automático em 24h e 48h.

3. Right to erasure (LGPD Art. 18):

- Edge Function `request-data-deletion`.
- Workflow: request → verificação email + confirmação 7 dias → soft delete + 30 dias window → hard delete cascade.
- Hard delete cascadeia: `profiles`, `student_question_attempts`, `subscriptions`, `essays`, `study_room_planner`, `consent_events` (mantém por obrigação legal 5 anos, anonimizado).

4. Right to portability:

- Edge Function `export-user-data`.
- Gera JSON + CSV de tudo que o usuário tem (profile, attempts, subscriptions, essays, planner).
- Email com signed URL R2 válida 24h.
- Registro em `consent_events`.

5. Política de retenção automatizada:

- Cron Supabase scheduled function `anonymize-old-logs`:
 - Logs > 90 dias: anonimizar `user_id` (hash).
 - `student_question_attempts` > 2 anos após cancelamento: deletar.
 - `consent_events` mantidos 5 anos.

6. Session Replay Sentry — LIBERADO após DPIA:

```
replaysSessionSampleRate: 0.05, // 5% das sessões
replaysOnErrorSampleRate: 1.0, // 100% dos erros
integrations: [
  ...,
  Sentry.replayIntegration({
    maskAllText: true,
    blockAllMedia: true,
    maskAllInputs: true,
    maskFn: (text) => '*'.repeat(text.length),
    // Bloquear replay para menores explicitamente
    beforeAddRecordingEvent: (event) => {
      if (isCurrentUserMinor()) return null;
      return event;
    },
  }),
],
```

7. Páginas legais:

- `/termos` — Termos de Uso versionados.
- `/privacidade` — Política de Privacidade + contato DPO.
- `/consentimento` — Gestão de consentimento (revogar, exportar, excluir).
- Signup exige aceite da versão ativa.

8. Response plan ANPD (runbook `stark/Stark/knowledge/rainha-runbook-anpd.md`):

- Passo-a-passo se ANPD abrir processo ou aluno reclamar.
- Prazos legais.
- Template de comunicação de incidente.

5.4.3 Deliverables da Fase 4

☐ DPIA concluído, revisado por advogado, arquivado.

- ☐ Termos + Privacy + Consentimento publicados e versionados.
- ☐ DPO nomeado, email público ativo.
- ☐ Schema + Edge Functions consentimento/exclusão/portabilidade em produção.
- ☐ Session replay Sentry ativo com masking agressivo + bloqueio para menores.
- ☐ Política de retenção rodando em cron.
- ☐ Runbook ANPD no vault.
- ☐ Re-consentimento base atual executado (opção A) ou comunicado (opção B).

5.4.4 Riscos e mitigações

- **Dayane resiste ao custo advogado:** apresentar como investimento — multa LGPD até R\$50M. Custo R\$5k é 0.01%.
- **Drop conversão -5 a -15%:** UX minimalista, email rápido ao responsável, reminder, resgate automático.
- **Re-consentimento confunde base atual:** campanha coordenada com Dayane, banner explicativo, suporte proativo.

FASE 5 — PERFORMANCE & ESCALA (mês 2, semanas 1–2)

Objetivo: validar que Rainha aguenta pico segunda 17h + crescimento até 800 alunos sem degradação.

5.5.1 Tarefas manuais do Mestre (~1h)

1. Aprovar upgrade Supabase se necessário (Pro US\$25 → Team US\$599, depende de uso real pós-Fase 2 métricas).
2. Aprovar Cloudflare R2 para imagens de questões (custo ~US\$15/1TB/mês vs Supabase Storage US\$25/TB + bandwidth).
3. Aprovar janela de manutenção para criação de índices (domingo madrugada, ~30min).

5.5.2 Tarefas automáticas do Claude Code (~20h)

1. **Load test k6 autenticado** `tests/load/peak-monday-17h.js` :

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 },
    { duration: '3m', target: 400 },
```

```

    { duration: '5m', target: 800 },
    { duration: '2m', target: 1600 }, // spike test
    { duration: '3m', target: 400 },
    { duration: '2m', target: 0 },
  ],
  thresholds: {
    'http_req_duration{name:answer_question}': ['p(95)<500', 'p(99)<1000'],
    'http_req_duration{name:load_caderno}': ['p(95)<800'],
    'http_req_duration{name:login}': ['p(95)<1000'],
    'http_req_failed': ['rate<0.01'],
    'checks': ['rate>0.99'],
  },
};

export function setup() {
  // Cria/reusa 1000 contas QA com is_qa_account=true
  return { qaAccounts: preloadQAAccounts(1000) };
}

export default function (data) {
  const account = data.qaAccounts[__VU % data.qaAccounts.length];

  // 1. Login
  const loginRes = http.post(`${SUPA_URL}/auth/v1/token?
grant_type=password`,
    JSON.stringify({ email: account.email, password: account.password }),
    { headers: { 'Content-Type': 'application/json', apikey: ANON_KEY },
tags: { name: 'login' } });
  check(loginRes, { 'login 200': r => r.status === 200 });
  const token = loginRes.json('access_token');

  // 2. Load caderno
  const cadernoRes = http.get(`${SUPA_URL}/rest/v1/questions?
select=...&limit=20`,
    { headers: { Authorization: `Bearer ${token}`, apikey: ANON_KEY },
tags: { name: 'load_caderno' } });
  check(cadernoRes, { 'caderno 200': r => r.status === 200 });

  // 3. Answer question (hot path - trigger on_answer_inserted roda aqui)
  const answerRes =
http.post(`${SUPA_URL}/rest/v1/student_question_attempts`,
    JSON.stringify({ question_id: pickQuestion(), is_correct: Math.random()
> 0.5, ... }),
    { headers: { Authorization: `Bearer ${token}`, apikey: ANON_KEY },
tags: { name: 'answer_question' } });
  check(answerRes, { 'answer 201': r => r.status === 201 });

```

```

    sleep(Math.random() * 3 + 1); // pensamento humano
}

```

2. Análise RLS com EXPLAIN ANALYZE:

- Script `scripts/rls-explain.mjs` gera EXPLAIN de queries equivalentes a cada policy em hot path.
- Tabelas auditadas: `questions`, `student_question_attempts`, `simulations`, `study_room_planner`, `subscriptions`, `ai_explanation_cache`.
- Relatório em `stark/Stark/AI-Brain/2026-05-rls-explain.md` com antes/depois.

3. Função cache de subscription (evita subquery correlata pesada):

```

CREATE OR REPLACE FUNCTION is_active_subscriber(uid UUID)
RETURNS BOOLEAN
LANGUAGE SQL
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
    SELECT EXISTS (
        SELECT 1 FROM subscriptions
        WHERE user_id = uid
            AND status = 'active'
            AND (expires_at IS NULL OR expires_at > now())
    );
$$;

-- RLS de questions usa essa função
DROP POLICY IF EXISTS "questions_select_active" ON questions;
CREATE POLICY "questions_select_active" ON questions
    FOR SELECT USING (is_active_subscriber(auth.uid()) OR
is_qa_account_fn(auth.uid()));

```

4. RLS de simulations:

```

CREATE POLICY "simulations_select_valid_window" ON simulations
    FOR SELECT USING (
        opens_at <= now() AND closes_at > now()
        AND assigned_group IN (
            SELECT cg.name FROM class_groups cg
            JOIN class_members cm ON cm.class_group_id = cg.id
            WHERE cm.student_id = auth.uid()
        )
    );

```

5. RLS de ai_explanation_cache:

```
DROP POLICY IF EXISTS "ai_cache_open" ON ai_explanation_cache;
CREATE POLICY "ai_cache_service_only" ON ai_explanation_cache
  FOR ALL TO service_role USING (true) WITH CHECK (true);
-- Usuários só acessam via Edge Function autenticada
```

6. Connection pooling:

- Migrar SUPABASE_DB_URL de PgBouncer session mode (port 5432) para Supervisor transaction mode (port 6543).
- Testar em staging primeiro (prepared statements incompatíveis em transaction mode — usar ?pgbouncer=true&statement_cache_size=0 ou equivalente no client).

7. Índices novos (com CREATE INDEX CONCURRENTLY em janela baixa):

```
CREATE INDEX CONCURRENTLY idx_attempts_user_created
  ON student_question_attempts(user_id, created_at DESC);

CREATE INDEX CONCURRENTLY idx_questions_active
  ON questions(subject, topic, difficulty) WHERE is_active = true;

CREATE INDEX CONCURRENTLY idx_subscriptions_active_user
  ON subscriptions(user_id) WHERE status = 'active';

CREATE INDEX CONCURRENTLY idx_consent_events_user_type
  ON consent_events(user_id, event_type);
```

8. Migração de imagens para Cloudflare R2:

- Script scripts/migrate-images-r2.mjs:
 - Lê questions com image_url no Supabase Storage.
 - Download → upload R2 com mesmo path.
 - Atualiza image_url para CDN Cloudflare.
 - Idempotente (pode rodar novamente; pula já migradas).
 - Mantém Supabase Storage como fallback por 30 dias.

9. ESLint rule require-channel-cleanup (fix INC-10):

- Detecta supabase.channel() dentro de componente React sem useEffect cleanup.

10. Audit e fix de trigger on_answer_inserted:

- Rever código do trigger.
- Adicionar teste integração.
- Garantir XP + streak atualizam em 100% dos casos.

5.5.3 Deliverables da Fase 5

- ☐ Load test atingindo 800 concurrent + spike 1600, p95 < 500ms em answer_question.
- ☐ Relatório EXPLAIN ANALYZE completo com antes/depois.

- ☐ Supervisor transaction mode ativo em produção.
- ☐ Índices criados sem lock contention.
- ☐ Imagens servindo do R2 com CDN Cloudflare.
- ☐ Zero realtime leaks em 48h monitoramento.
- ☐ Trigger `on_answer_inserted` com teste verde.

5.5.4 Riscos e mitigações

- **Supervisor quebra prepared statements:** testar staging, client config, rollback fácil (volta para session mode).
 - **Índice em prod trava:** sempre `CONCURRENTLY`, janela domingo madrugada, alerta ativo.
 - **Migração imagens quebra URLs:** Supabase Storage mantido 30 dias, rollback via script reverso.
 - **Load test contamina dados:** `is_qa_account=true` + RLS filtrando em métricas de negócio (Fase 3).
-

FASE 6 — FEATURE FLAGS & GOVERNANCE (mês 2, semanas 3–4)

Objetivo: desacoplar deploy de release + reduzir SPoF José.

5.6.1 Tarefas manuais do Mestre (~3h + 16h pair programming)

1. **Contratar VPS para PostHog self-hosted:**
 - Hetzner CX21 (€5.83/mês, 4GB RAM) ou Contabo VPS S (€6.99/mês, 8GB RAM).
 - Ubuntu 22.04.
 - Enviar credenciais SSH ao Claude Code via canal seguro.
2. **Domínio** `posthog.foryoucode.app` **apontado para VPS** (A record + Cloudflare proxy).
3. **4 sessões de pair programming com José (4h/semana por 4 semanas = 16h):**
 - Mestre presente em sessões críticas.
 - Foco: Mestre entender fluxos críticos para dar primeiro diagnóstico P0.
 - Gravar sessões (consentimento José), armazenar em `stark/Stark/AI-Brain/pair-sessions/`.

5.6.2 Tarefas automáticas do Claude Code (~16h)

1. **PostHog self-hosted:**
 - Docker Compose: PostgreSQL + ClickHouse + Redis + PostHog app.
 - Reverse proxy Caddy com Let's Encrypt automático.
 - Backup diário (DB + ClickHouse snapshot) → R2.

- Alertas disk > 80% → Discord.

2. Integração PostHog no Rainha:

- `src/lib/posthog.ts` inicialização LGPD-safe (sem autocapture de inputs, opt-out via flag perfil).
- Hook `useFeatureFlag(key, defaultValue)` wrapper.
- `track(event, props)` helper com sanitização PII.

3. Feature flags iniciais:

- `new-caderno-ui`
- `ai-explanation-v2`
- `advanced-planner`
- `kill-switch-eduzz`
- `kill-switch-signup`
- `kill-switch-ai`

4. Runbook `stark/Stark/knowledge/rainha-runbook.md` (anexo I):

- Incidente: usuário não consegue logar → checar Auth logs, JWT, rate limit.
- Incidente: pagamento Eduzz não confirmado → checar webhook logs, idempotency table, reprocessar manualmente.
- Incidente: questão não carrega → checar `questions` RLS, subscription ativa, CDN R2.
- Incidente: banco lento → checar Supabase dashboard, connections, slow queries.
- Incidente: deploy quebrou prod → `git revert` + redeploy + postmortem.
- Incidente: `ChunkLoadError` em massa → checar cache Cloudflare, purge, verificar deploy consistency.
- Para cada: primeiro diagnóstico, ações, quando escalar.

5. Documentação técnica completa no vault:

- `rainha-architecture.md` — diagrama mermaid + descrição camadas.
- `rainha-edge-functions.md` — cada function: payload, triggers, idempotência.
- `rainha-rls.md` — cada tabela, policies, razão.
- `rainha-deployment.md` — como deployar manual, como rollback.
- `rainha-onboarding-dev.md` — como novo dev sobe projeto em 1h.

6. Automação onboarding:

- `scripts/bootstrap-dev.sh` — clone, install, env setup, primeiro build.
- `README.md` com quickstart.

7. 3 simulações de incidente P0 sem José:

- Simulação 1: "Usuário premium não consegue ver questões" (RLS).
- Simulação 2: "Pagamento Eduzz perdido" (webhook).
- Simulação 3: "Deploy quebrou login" (rollback).

- Mestre executa seguindo runbook. Mede: tempo até diagnóstico correto, tempo até mitigação.

5.6.3 Deliverables da Fase 6

- ☐ PostHog self-hosted rodando, Rainha enviando eventos.
- ☐ 6 feature flags ativos incluindo 3 kill switches.
- ☐ Runbook completo, testado em 3 simulações (Mestre executa sem José).
- ☐ Documentação técnica completa no vault.
- ☐ 4 sessões pair programming realizadas e gravadas.

5.6.4 Riscos e mitigações

- **PostHog self-hosted exige maintenance:** backup + script upgrade mensal + alerta disk.
 - **José resiste à perda de control único:** framing "liberdade para tirar férias sem app cair".
-

FASE 7 — MIGRAÇÃO LOVABLE CLOUD → SUPABASE DIRETO (mês 3–5)

Objetivo: eliminar lock-in Lovable Cloud. Rainha 100% em Supabase nativo.

5.7.1 Tarefas manuais do Mestre (~4h)

1. **Criar projeto Supabase direto:**
 - `rainha-prod-nativo` fora do Lovable.
 - Plano Pro ou Team (baseado em uso real pós-Fase 5).
 - Anotar credenciais.
2. **Negociar com Lovable (se necessário):**
 - Exportação completa dados + schema (geralmente sem custo).
3. **Aprovar janela cutover:**
 - Domingo madrugada (03h–05h BRT).
 - Comunicação prévia 48h: banner + email Dayane → alunos.

5.7.2 Tarefas automáticas do Claude Code (~40h ao longo de 8 semanas)

Semanas 1–2: Staging Supabase direto

1. Provisionar `rainha-staging-nativo` com mesmo schema.
2. Dump schema Lovable → apply staging nativo.
3. Clonar dados sintéticos (não PII real).

4. Edge Functions via Supabase CLI (não Lovable).
5. Rainha staging apontando para staging-nativo.

Semanas 3–4: Testes de paridade

1. Regression suite completa contra staging-nativo.
2. Load test contra staging-nativo.
3. Checklist paridade: cada feature comparada prod Lovable vs staging-nativo.
4. Edge cases: realtime, triggers, RLS complexo, AI Gateway.
5. **Se Lovable AI Gateway for exclusivo:** migrar para OpenAI/Anthropic/Google direto (código já prevê via env var).

Semana 5: Replicação lógica em paralelo

1. Configurar publication Lovable (primary) + subscription Supabase direto (replica).
2. Monitorar lag 7 dias. Meta: < 5s.
3. Comparar row counts hourly. Divergência > 0.01% = abort.

Semana 6: Cutover (janela domingo madrugada)

1. **T-60min:** banner "manutenção 30min" no Rainha.
2. **T-10min:** freeze writes Lovable via kill switch.
3. **T-5min:** verificar replicação em zero lag.
4. **T-0:** promover Supabase direto para primary.
5. **T+5min:** atualizar env vars Cloudflare Pages.
6. **T+10min:** redeploy frontend.
7. **T+15min:** smoke test pós-cutover.
8. **T+20min:** descongelar writes.
9. **T+30min:** remover banner.
10. **T+0 a T+6h:** monitoramento intensivo.

Semana 7: Lovable como backup read-only

1. Lovable mantido como cópia read-only 30 dias.
2. Qualquer issue → rollback documentado (reverter credenciais).

Semana 8: Corte definitivo

1. Cancelar assinatura Lovable Cloud.
2. Arquivar projeto Lovable.
3. José passa a usar Lovable como **IDE frontend apenas** — backend 100% via Supabase CLI + MCP.

4. Economia Lovable reinvestida em Supabase tier maior + R2 + VPS.

5.7.3 Deliverables da Fase 7

- ☐ Rainha 100% Supabase direto por > 7 dias sem regressão.
- ☐ Lovable Cloud desativado.
- ☐ José treinado em Supabase CLI + MCP.
- ☐ Documentação atualizada refletindo nova arquitetura.
- ☐ Postmortem da migração no vault.

5.7.4 Rollback

- Semanas 5–6 (paralelo): reverter credenciais = volta Lovable em < 15min.
- Semana 7 (paralelo read-only): rollback requer recriar writes em Lovable, custoso mas possível (< 4h).
- Após semana 8: rollback inviável (por isso 30 dias de paralelo).

6. TAREFAS MANUAIS DO MESTRE — LISTA CONSOLIDADA

Tudo que exige acesso externo, decisão, assinatura ou pagamento — só você pode fazer.

Fase 0 (48h)

- ☐ Bucket Cloudflare R2 criado
- ☐ Tokens R2 gerados (R2_ACCESS_KEY_ID, SECRET, ACCOUNT_ID)
- ☐ PITR habilitado Supabase
- ☐ Credenciais Supabase coletadas (Service Role, DB URL, Project Ref)
- ☐ GitHub Secrets iniciais configurados (6 secrets)
- ☐ Credenciais enviadas ao Claude Code

Fase 1 (semana 1)

- ☐ Cloudflare Pages conectado ao GitHub
- ☐ Env vars Cloudflare Pages configuradas
- ☐ DNS apontado para Cloudflare após 48h estável
- ☐ Cloudflare Security ativado (WAF, Bot Fight, Rate Limit, SSL Full Strict)

Fase 2 (semana 2)

- ☐ Conta Sentry criada (org foryou-code, projeto rainha)

- ☐ GitHub Secrets: SENTRY_DSN, SENTRY_AUTH_TOKEN, SENTRY_ORG, SENTRY_PROJECT
- ☐ Canal Discord [#rainha-alerts](#) + webhook
- ☐ (Opcional) Conta Checkly

Fase 3 (semana 3)

- ☐ GitHub branch protection main configurada

Fase 4 (semana 4)

- ☐ Advogado LGPD contratado (R\$2k–5k)
- ☐ DPIA revisado com advogado
- ☐ DPO nomeado, email [dpo@...](#) criado
- ☐ Dayane alinhada sobre fluxo consentimento
- ☐ Decisão re-consentimento base atual (A ou B)

Fase 5 (mês 2, sem 1–2)

- ☐ Upgrade Supabase aprovado (se necessário)
- ☐ Cloudflare R2 para imagens aprovado
- ☐ Janela manutenção índices aprovada (domingo madrugada)

Fase 6 (mês 2, sem 3–4)

- ☐ VPS Hetzner/Contabo contratado + SSH keys enviadas
- ☐ Domínio posthog.foryoucode.app apontado
- ☐ 16h pair programming com José (4 sessões)

Fase 7 (mês 3–5)

- ☐ Projeto Supabase direto criado (rainha-prod-nativo)
- ☐ Janela cutover definida (domingo madrugada específico)
- ☐ Comunicação 48h pré-cutover (banner + email Dayane)

Contínuas (toda semana)

- ☐ Revisar relatório semanal Claude Code no vault (~15min)
- ☐ Aprovar mudanças de config que afetam custo
- ☐ Aprovar qualquer ação real externa via CONFIRMO (regra CLAUDE.md ForYou)

Estimativa total horas manuais Mestre (6 meses): 35–45h (média ~2h/semana).

7. TAREFAS AUTOMÁTICAS DO CLAUDE CODE — LISTA CONSOLIDADA

Tudo que eu executo sem depender de você (exceto CONFIRMO quando ação real externa):

Código:

- vite.config, ErrorBoundary, Sentry SDK, PostHog SDK
- CLAUDE.md do repo Rainha
- ESLint plugin custom (`eslint-plugin-rainha-blindagem`)
- Husky + lint-staged
- Hooks React (useFeatureFlag, useTracking)

Infraestrutura:

- GitHub Actions workflows (CI 8 gates, deploy, backup, verify)
- Scripts backup manual/automated/verify
- Docker Compose PostHog
- Caddy reverse proxy
- Migration scripts imagens R2

Banco:

- Migrations RLS, DPIA schema, índices
- Função `is_active_subscriber()` + outras helpers
- Edge Functions: verify-guardian-consent, request-data-deletion, export-user-data, anonymize-old-logs, health
- Auditoria EXPLAIN ANALYZE

Testes:

- Playwright regression suite (30 testes)
- Vitest unit tests
- Edge Functions contract tests (Deno)
- k6 load test autenticado

Documentação:

- Runbook completo
- Documentação técnica (architecture, edge-functions, rls, deployment, onboarding-dev)
- Relatórios semanais no vault

- Postmortem de incidentes
- DPIA rascunho

Migração Lovable → Supabase direto:

- Replicação lógica setup
- Cutover scripts
- Rollback scripts

Monitoramento:

- Sentry alerts config
- Cloudflare Analytics dashboard
- PostHog dashboards

Estimativa total Claude Code (6 meses): 130–160h de trabalho automatizado.

8. GATILHOS DE AÇÃO E ROLLBACK

8.1 Gatilhos de saída antecipada Lovable Cloud (pula para Fase 7)

Qualquer um dispara migração imediata:

- Lovable aumenta preço > 30%
- SLA degrada > 1% incidentes/mês afetando Rainha
- Lovable desconecta GitHub repo novamente
- Lovable corrompe schema ou dados
- Rainha passa de 500 alunos ativos antes do mês 3
- Feature crítica não suportada pela Lovable

8.2 Gatilhos de pause do plano

Qualquer um dispara pause + reavaliação:

- José ausente > 15 dias úteis
- Incidente P0 aberto não resolvido em 24h
- Mudança significativa de prioridade do negócio (Mestre ou Dayane)
- Dayane solicita mudança de escopo crítica

8.3 Kill switches (incidente crítico)

- `kill-switch-eduzz` — desliga webhook Eduzz
- `kill-switch-signup` — bloqueia novos cadastros
- `kill-switch-ai` — desliga AI Gateway
- Revert Cloudflare Pages para deploy anterior (1-click via dashboard)
- `git revert <sha>` + redeploy (< 5min)

8.4 Rollback por fase (resumo)

- **Fase 0–2:** 100% reversível sem perda.
- **Fase 3–4:** reversível com esforço (schema DPIA é aditivo).
- **Fase 5:** reversível com monitoring apertado (índices drop + pooling revert).
- **Fase 6:** reversível com custo (PostHog desligar).
- **Fase 7:** reversível apenas em janela 30 dias (paralelo read-only).

9. CUSTOS E INVESTIMENTO

9.1 Custos recorrentes mensais

Item	Hoje	Pós-plano (mês 6)	Δ
Lovable Cloud (bundled Supabase + AI Gateway)	~US\$49	0	-49
Supabase Pro direto	0	US\$25	+25
Cloudflare Pages	0 (via Vercel)	0	0
Vercel	US\$0-20	0	-0/20
Cloudflare R2 (imagens)	0	~US\$15	+15
Sentry (Team após escalar)	0	US\$0-26	+0/26
VPS Hetzner (PostHog)	0	US\$7	+7
Checkly (opcional)	0	US\$0-40	+0/40
OpenAI/Anthropic direto (se Lovable AI sair)	0	US\$20-100 (estimado)	+20/100
Total mensal	~US\$49-69	US\$47-213	-2 a +144

Custo subirá proporcional ao uso real. Pior caso US\$213/mês = US\$2.556/ano. Comparado ao ticket Rainha (R\$27k entregue + potencial recorrente), é marginal.

9.2 Custos one-shot

Item	Custo
Advogado LGPD (DPIA + termos)	R\$2.000–5.000
Horas Mestre (35–45h em 6 meses)	Custo oportunidade
Horas Claude Code (130–160h)	Incluído em assinatura Claude Max
Horas José (ajustes pontuais + pair)	Incluído (equipe)

9.3 ROI esperado

Risco evitado:

- Multa LGPD até R\$50M ou 2% faturamento.
- Perda total banco (unrecoverable) = fim do projeto.
- Perda de clientes por incidente recorrente.
- Processo ANPD.

Ganho direto:

- Capacidade escalar 202 → 800+ alunos sem refatorar.
- Redução custo operacional (R2 vs Supabase Storage, Cloudflare vs Vercel).
- Independência Lovable (negociação futura).

Ganho indireto:

- Mestre deixa de apagar incêndio → mais tempo para vender ForYou Code.
- José não é indispensável → sustentabilidade da equipe.
- Rainha vira case replicável (white-label ForYou Code).

10. MATRIZ DE RISCOS

#	Risco	Probabilidade	Impacto	Mitigação	Fase
1	Perda dados sem backup	Alta (se nada feito)	Catastrófico	Backup diário + PITR + verify	0
2	Multa LGPD	Média	Alto (R\$ milhões)	DPIA + consent + retenção	4

#	Risco	Probabilidade	Impacto	Mitigação	Fase
3	Lovable lock-in	Certo	Médio-alto	Plano saída com gatilhos	7
4	José indisponível	Média	Alto	Runbook + pair + docs	6
5	Pico segunda 17h crash	Média	Médio	Load test + pooling + índices	5
6	Custo Supabase estoura	Baixa	Médio	Índices + pooling + monitoring	5
7	Migração cutover falha	Baixa	Alto	Paralelo 30 dias + rollback	7
8	Regressão pós-deploy	Baixa (com CI)	Baixo-médio	8 gates + feature flags	1–3
9	Advogado LGPD demora	Média	Baixo	Começar Fase 4 cedo	4
10	Alunos resistem consentimento	Média	Baixo	UX + Dayane comunica	4
11	Sentry free estoura	Baixa	Baixo	Sampling + alertas	2
12	Playwright flaky CI	Média	Baixo	Retries + determinismo	3
13	PostHog VPS cai	Baixa	Baixo	Backup + alert disk	6
14	Cloudflare Pages outage	Muito baixa	Alto	DNS revert Vercel (fallback)	1

11. CRONOGRAMA VISUAL DETALHADO

2026-04-18 (hoje) _____

ABR Sem 1 [FASE 0] Backup imediato (48h)  CRÍTICA
[FASE 1] CLAUDE.md + Cloudflare Pages + ErrorBoundary

ABR Sem 2 [FASE 2] Sentry + Cloudflare Analytics + Discord alerts

ABR Sem 3 [FASE 3] CI 8 gates + staging + regression suite

ABR Sem 4 [FASE 4] DPIA + consent flow + session replay liberado

MAI Sem 1-2 [FASE 5] k6 load test + RLS optimize + R2 images

MAI Sem 3-4 [FASE 6] PostHog + runbook + pair programming

JUN

JUL [FASE 7] Migração Lovable → Supabase direto

AGO

SET 2026

Rainha 100% Supabase direto

800 alunos suportados

Métricas no verde

Lovable cortado

Mestre dorme tranquilo

12. COMO COMEÇAR — PRÓXIMOS PASSOS

Hoje mesmo

1. Leia este documento inteiro (você está quase no fim).
2. Responda **"CONFIRMO FASE 0"** para eu iniciar backup imediato (pg_dump agora + GitHub Actions).
3. Execute suas tarefas manuais da Fase 0 (lista seção 6, ~2h).
4. Envie credenciais conforme lista via canal seguro.

Esta semana

- Fase 0 concluída (48h).
- Fase 1 em paralelo (CLAUDE.md, Cloudflare Pages, ErrorBoundary).

Cadência de revisão

- **Semanal (30min):** eu envio relatório em stark/Stark/AI-Brain/rainha-relatorio-sem-XX.md . Você lê e valida.
- **Mensal (1h):** revisão completa + ajuste de prioridade se necessário.
- **Por fase (ao concluir DoD):** você valida checklist + autoriza avanço para próxima fase.

13. PALAVRA FINAL HONESTA

Este plano **não** transforma Rainha em SaaS enterprise-grade para 10.000 alunos em 6 meses. Isso seria mentira.

Este plano **transforma** Rainha em:

- SaaS bem construído para 200–800 alunos.
- Preparado para crescer até 1.600–2.000 sem refatoração crítica.
- Sem dor jurídica (LGPD/ECA conformado).
- Sem José indispensável.
- Sem Lovable como camarote obrigatório.

Se executado integralmente:

- **Nota técnica final:** 8.5/10 (código vivo nunca atinge 10).
- **Nota de risco:** 1/10 (baixíssimo).
- **Nota de preparação para 1.600:** 9/10 (algumas otimizações extras passando de 1.200).

Se executado parcialmente:

- **Fase 0 sozinha:** evita catástrofe de perda total.
- **Fases 0–2:** evita 80% dos bugs recorrentes.
- **Fases 0–4:** Rainha legalmente defensável.
- **Fases 0–6:** Mestre dorme tranquilo, José pode tirar férias.
- **Fases 0–7:** Rainha pronto para venda, investimento ou continuidade sem você.

O piso é você escolher. Eu executo até onde você autorizar.

14. ANEXOS (serão criados na fase correspondente, não neste documento)

- **Anexo A:** `CLAUDE.md` do repo Rainha (Fase 1).
- **Anexo B:** `eslint-plugin-rainha-blindagem` completo (Fase 1).
- **Anexo C:** `.github/workflows/ci.yml` + `deploy.yml` + `backup-*.yaml` (Fases 0, 1, 3).
- **Anexo D:** `scripts/backup-manual.sh`, `scripts/rls-explain.mjs`, `scripts/migrate-images-r2.mjs` (Fases 0, 5).
- **Anexo E:** `tests/load/peak-monday-17h.js` k6 (Fase 5).
- **Anexo F:** `tests/regression/bug-*.spec.ts` Playwright 30 testes (Fase 3).
- **Anexo G:** Migrations SQL RLS + DPIA schema + índices (Fases 4, 5).
- **Anexo H:** DPIA template preenchido (Fase 4).

- **Anexo I:** rainha-runbook.md (Fase 6).
- **Anexo J:** Documentação técnica completa (Fase 6).

Cada anexo é gerado automaticamente por Claude Code na fase correspondente, commitado no repo ou salvo no vault Stark.

Documento gerado por Claude Opus 4.7

Versão 2.0 — 2026-04-18

Próxima revisão obrigatória: 2026-05-18 (pós-Fase 4)

Para iniciar: responda CONFIRMO FASE 0 nesta conversa.